

UNIVERSITY OF BUEA

FACULTY OF
ENGINEERING AND
TECHNOLOGY
(FET)

PO BOX 63



REPUBLIC OF CAMEROON

PEACE-WORK-
FATHERLAND

BUEA, SOUTH WEST REGION

TASK 3 REPORT

Requirement Analysis For a Crowdsensed Drive Test Mobile Application for Coverage Prediction

COURSE TITLE: ***INTERNET PROGRAMMING AND
MOBILE PROGRAMMING***

COURSE CODE: ***CEF 440***

COURSE INSTRUCTOR: ***DR NKAMENI VALERY***

➤ GROUP 13 MEMBERS

1. IMELE AZAFA JOSE DE KAJOS -----FE23A068
2. AYUK AMELIA AGBOR OROCK----- FE23A020
3. NGOME FERNANDEZ MANYAKA----- FE23A111
4. TADJEU NGNINTEDEM AUBRY-JUNEX----- FE23A148
5. BESSONG TAKANG TABOT WILLY----- FE23A025

DEPARTMENT OF COMPUTER ENGINEERING

ACADEMIC YEAR 2025 - 2026

Table of Contents

INTRODUCTION.....	1
1. Requirement Review and Analysis.....	2
1.1 Completeness.....	2
1.2 Clarity	2
1.3 Technical Feasibility	3
1.4 Dependency Relationships.....	4
2. Inconsistencies, Ambiguities, and Missing Information.....	6
2.1 Inconsistencies Found.....	6
2.1.1 Battery Efficiency vs Real-Time Data Collection.....	6
2.1.2 WiFi-Only Upload vs Live Coverage Map.....	7
2.1.3 Full Anonymity vs Device Consistency for ML	7
2.1.4 Collection Timing Preference	7
2.2 Ambiguities Resolved.....	7
2.3 Missing Information	8
2.3.1 Mobile Application Gaps.....	8
2.3.2 Backend Server Gaps.....	9
2.3.3 ML Coverage Prediction Gaps	9
3. Requirement Prioritization	10
3.1 Evidence Base	11
3.1.1 Survey Findings (100 Smartphone Users, Buea).....	11
3.1.2 Engineer Interview Findings (MTN & Orange).....	11
3.1.3 Reverse Engineering (OpenSignal, Network Cell Info Lite).....	11
3.2 Priority Matrix	12
3.3 Summary by Priority	14
3.4.1 Must Have Requirements.....	14
3.4.2 Should Have Requirements.....	14
3.4.3 Could Have Requirements	15
3.4.4 Won't Have (Current Phase)	15
4. Requirement Classification	16
Classification Framework	16
4.1 Functional Requirements	16
4.2 Non-Functional Requirements.....	17

5. Software Requirements Specification (SRS)	19
5.1 Introduction	19
5.1.1 Purpose	19
5.1.2 Document Conventions	19
5.1.3 Intended Audience	19
5.1.4 Project Scope	19
5.1.5 Key Definitions	20
5.2 Overall Description	20
5.2.1 Product Perspective	20
5.2.2 User Classes	20
5.2.3 Operating Environment	21
5.2.4 Constraints	21
5.2.5 Assumptions and Dependencies	21
5.3 System Features	21
5.3.1 Functional Requirements	21
5.4 External Interface Requirements	22
5.4.1 User Interface	22
5.4.2 Hardware Interfaces	22
5.4.3 Software Interfaces	22
5.4.4 Communication Interfaces	22
5.5 Non-Functional Requirements	22
5.5.1 Performance	22
5.5.2 Resource Constraints	22
5.5.3 Security	23
5.5.4 Reliability	23
5.5.5 Usability	23
5.5.6 ML Accuracy	23
6. Validation with Stakeholders	24
6.1. Introduction	24
6.2. Why Validation is Important	24
6.3. Stakeholders Validation Methods	25
6.4. Validation Techniques	26
6.5. Validation checklist	27
6.6 Validation Outcomes	28
CONCLUSION	29

INTRODUCTION

This report presents the full requirement analysis for CrowdSenseNet: a Crowdsensed Drive Test Mobile Application designed to solve the problem of inconsistent mobile network coverage in Cameroon. The work builds directly on the requirements gathered in Task 2, where the team collected information through four methods: surveys of 102 smartphone users, interviews with network engineers at MTN and Orange, team brainstorming sessions, and reverse engineering of existing apps such as OpenSignal and Network Cell Info Lite.

Task 3 takes all of that raw material and puts it through a structured analysis process. We reviewed the requirements for completeness, clarity, technical feasibility, and dependencies. We found and resolved inconsistencies, cleared up ambiguities, and filled in missing information. We then prioritised the requirements using the MoSCoW method, classified them into functional and non-functional groups, and brought them together into a full Software Requirements Specification (SRS). The final step was validating everything with the stakeholders who will use, operate, and oversee the system.

The goal of all this work is simple: before anyone writes a single line of code, we want to be confident that we know exactly what we are building, why we are building it, and that the people it affects have agreed that it meets their needs.

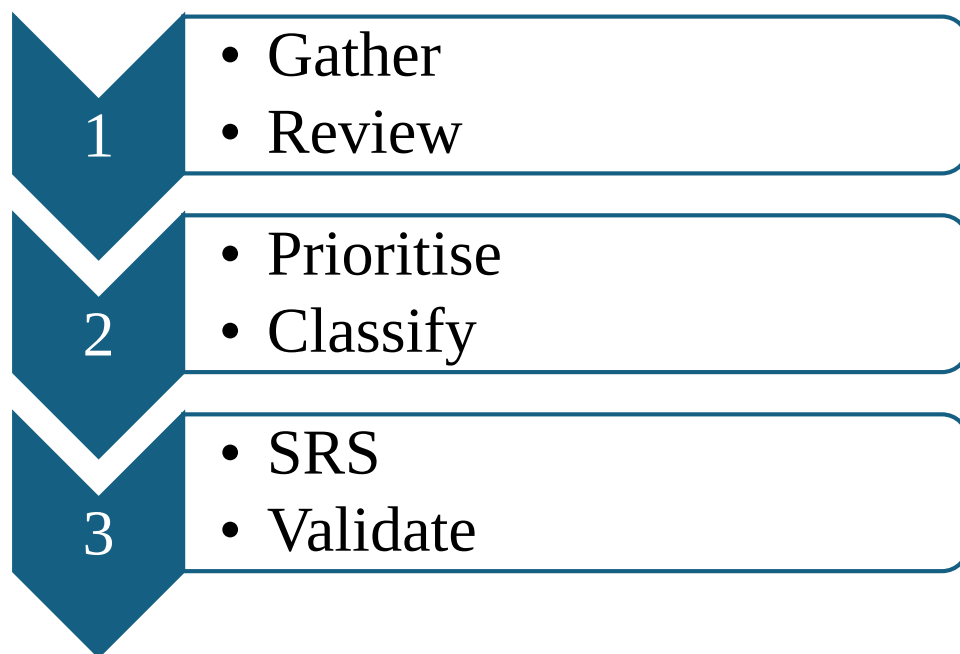


Figure 1: Overview of the Task 3 requirement analysis process

1. Requirement Review and Analysis

We obtained requirements from four sources in Task 2: MNO engineer interviews, a user survey with 102 responses, team brainstorming sessions, and reverse engineering of OpenSignal and NetMonster. Before using those requirements to design anything, we reviewed them across four areas: completeness, clarity, technical feasibility, and dependency relationships.

1.1 Completeness

When checking for completeness, we made sure that no stakeholder need was left without a corresponding requirement. Based on the gathered information, the following areas were well covered:

- RF parameters: RSRP, SINR, RSRQ, RSSI, and throughput
- Privacy and data anonymity
- Battery efficiency
- User on/off toggle for data collection
- WiFi-only upload option

A few areas needed more detail:

- MNO interview data was partially blocked for confidentiality reasons. Some RF parameter requirements were inferred rather than formally confirmed, leaving room for further clarification.
- The GPS accuracy threshold appeared as an implied requirement in the data rather than a clearly written one. It has since been formalised.

1.2 Clarity

We identified requirements that were too vague or subjective and replaced them with clear, measurable versions. The table below shows the original wording alongside the improved version.

Original (Ambiguous)	Improved (Clear and Measurable)
App must use minimal battery	App background sensing must not increase battery drain by more than 5% per hour
Simple, clean interface usable by non-technical users	App must require no more than 3 steps to enable or disable data collection

Original (Ambiguous)	Improved (Clear and Measurable)
System must support data collection in rural areas	System must store data locally when connectivity is unavailable and sync automatically when reconnected
Good GUI	App shall have 4 main screens: Home, Live Map, History, Settings. Key actions reachable within 3 taps. Supports English and French
Anonymous data	App shall not collect name, email, phone number, IMEI, or IP address
Collect signal data	App collects RSRP, RSRQ, SINR, RSSI, Cell ID, network type, GPS, timestamp, and anonymous token every 10 seconds while moving
Predict coverage holes	Coverage hole defined as predicted RSRP below -110 dBm
Works offline	Data collection, local storage, and history viewing shall work offline

Table 1.1: Original ambiguous requirements and their clarified replacements

1.3 Technical Feasibility

Each major requirement was assessed and rated as feasible, partially feasible, or not feasible within the current project scope.

Requirement	Rating	Reason
Collect RF parameters via Android API	Feasible	Android TelephonyManager API supports RSRP, SINR, RSRQ, and RSSI directly
GPS tagging per measurement	Feasible	Standard Android Location API with FusedLocationProvider
Offline local storage and auto upload	Feasible	SQLite via Room DB, background sync via WorkManager

Requirement	Rating	Reason
Map-based heatmap visualisation	Partially feasible	Requires backend integration and a mapping library; achievable but complex
Multi-operator support (MTN and Orange)	Feasible	Android API automatically detects the active operator
GeoJSON export	Partially feasible	Standard format but requires backend development effort
ML-based coverage prediction	Not feasible (now)	Needs training data from a live pipeline; deferred to a later phase

Table 1.2: Technical feasibility assessment for major requirements

1.4 Dependency Relationships

Some requirements only make sense if others are already in place. We mapped those dependencies to make sure nothing gets built out of order.

Requirement	Depends On	Why
Auto upload to server	Offline local storage	Data must be stored locally before it can be uploaded
Map-based visualisation	GPS tagging and backend API	Cannot plot coverage without coordinates and a data endpoint
WiFi-only upload setting	Auto-upload feature	This is a sub-feature of the upload logic
Coverage hole flagging (RSRP below -110 dBm)	RSRP collection	Cannot flag what has not been collected
Heatmap	GeoJSON export and map visualisation	Backend must produce GeoJSON for the map to consume

Table 1.3: Requirement dependency relationships

Requirement Dependency Map

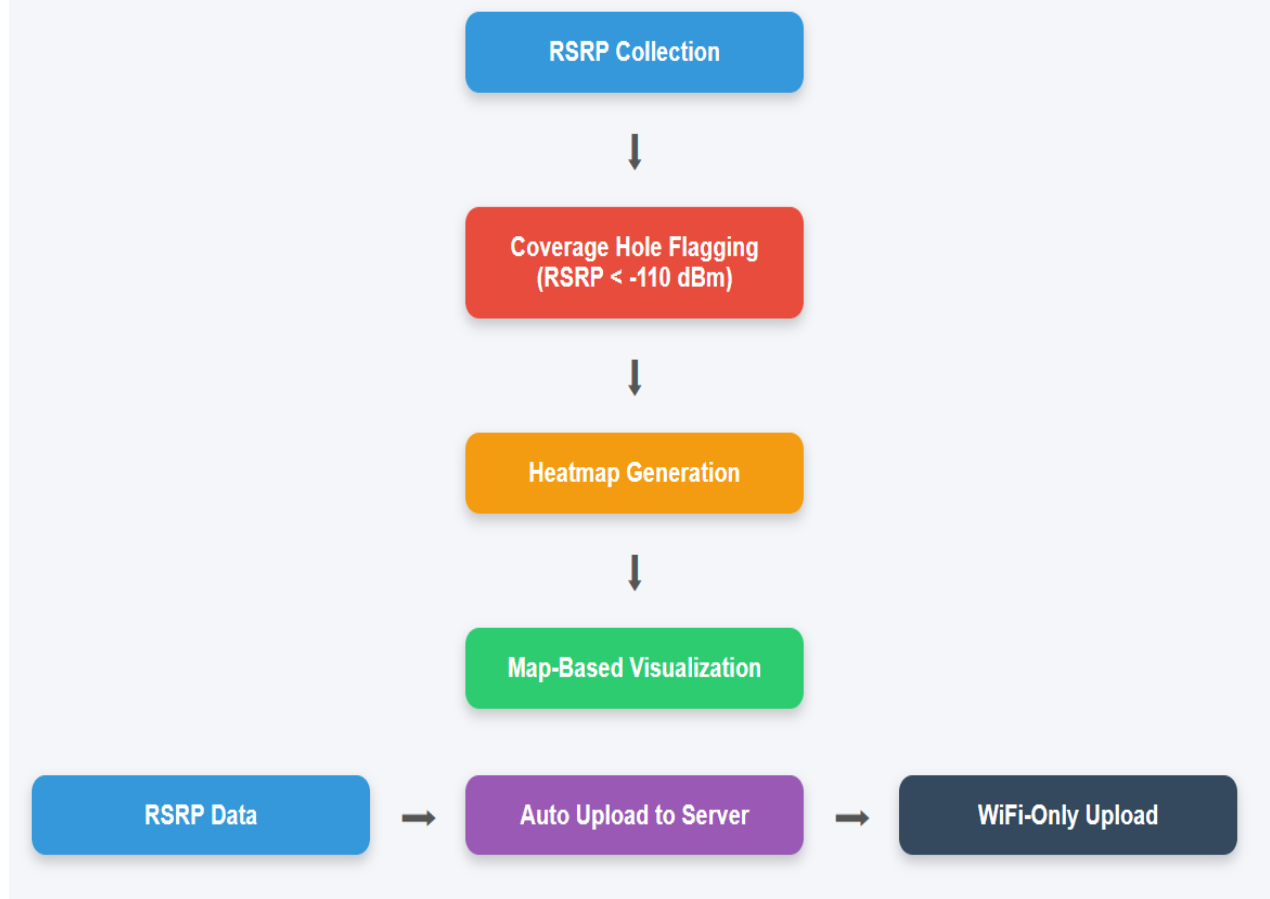


Figure 2: Requirement dependency map

2. Inconsistencies, Ambiguities, and Missing Information

In this task, the requirements gathered for the Crowdsensed Drive Test Mobile Application were analysed to identify inconsistencies, ambiguities, and missing information. This analysis ensures that the proposed system is clear, complete, reliable, and suitable for real-world deployment. By resolving these issues early, the development team can reduce errors, improve system quality, and ensure that the final MVP effectively supports mobile network coverage prediction in Cameroon.

MVP Scope

The Minimum Viable Product (MVP) covers:

- signal data collection (RSRP, RSRQ, SINR, GPS)
- local offline storage
- backend data upload
- basic coverage map
- ML coverage hole prediction.

2.1 Inconsistencies Found

An inconsistency is where two requirements ask for things that conflict with each other. We found four.

2.1.1 Battery Efficiency vs Real-Time Data Collection

Conflict: Users want real-time continuous data collection AND minimal battery usage at the same time.

Source: Survey data shows 64.8% experience issues very often, while 37.4% listed minimal battery as a condition for participating.

Why it matters: Keeping the CPU and GPS radio active continuously drains battery. These two requirements directly oppose each other.

Resolution: Smart adaptive sampling. The app collects data every 10 seconds when the device is moving (speed above 5 km/h) and pauses when stationary for more than 30 seconds. Users can adjust the interval between 5 and 30 seconds in Settings.

2.1.2 WiFi-Only Upload vs Live Coverage Map

Conflict: 56% of users want uploads only on WiFi, but the app also promises a live coverage map that needs fresh data.

Source: Survey shows 56% want data saving conditions, while 12.1% are motivated by seeing a live map.

Resolution: Data is always stored locally first. WiFi-only upload is the default. Users can enable mobile data upload in Settings. The map refreshes whenever new data reaches the server.

2.1.3 Full Anonymity vs Device Consistency for ML

Conflict: Users want fully anonymous data, but the ML model performs better when it can link readings from the same device over time.

Resolution: The app generates a random anonymous UUID on first launch. This is stored locally and sent with each reading. It allows device-consistent data without collecting any personal information.

2.1.4 Collection Timing Preference

Conflict: 46% of users want data collected only when the app is open, while 36% want always-on background collection. Both cannot be the single fixed default.

Resolution: Both modes are available as a user-controlled setting. The default is background mode (always-on). A Settings toggle lets users switch to collect-only-when-app-is-open mode.

2.2 Ambiguities Resolved

Ambiguous requirements are those where the wording is unclear or means different things to different people. We identified seven and clarified each one. These are already captured in the clarity table in Section 1.2.

Ambiguous Requirement	The Problem	Clarified Requirement
“Good GUP”	Too vague	App shall have 4 main screens: Home, Live Map, History, Settings. Key actions reachable within 3 taps. Supports English and French.
“Minimal battery usage”	No defined threshold	App shall consume less than 5% battery per hour during active background collection.
“Anonymous data”	Unclear definition	App shall not collect name, email, phone number, IMEI, or IP address.

“Works offline”	Which features offline?	Data collection, local storage, and history viewing shall work offline.
“Secure” / “Safe data”	Security not specified	Data transmitted using HTTPS/TLS 1.2+.
“Collect signal data”	Which parameters?	App collects RSRP, RSRQ, SINR, RSSI, Cell ID, network type, GPS, timestamp, anonymous token every 10 seconds while moving.
“Predict coverage holes”	Coverage hole undefined	Coverage hole defined when predicted RSRP < -110 dBm.

2.3 Missing Information

After reviewing all gathered requirements, we found 11 gaps where important scenarios had not been addressed at all. These have been added as new requirements.

2.3.1 Mobile Application Gaps

Gap Identified	New Requirement Added
Minimum Android version not specified	App supports Android 8.0 and above (API level 26)
GPS disabled scenario not handled	Pause collection and notify the user when GPS is turned off
Permission denial not handled	Show explanation and disable the affected feature gracefully
Local storage limit not defined	Delete oldest uploaded readings automatically after 50MB
Battery saver mode behaviour not defined	Request battery optimization exemption from the system

Table 2.1: Missing mobile app requirements and their additions

2.3.2 Backend Server Gaps

Gap Identified	New Requirement Added
Server downtime handling not defined	Retry uploads automatically every 30 minutes
Geographic boundary not defined	Reject readings that fall outside Cameroon's bounding box
Duplicate GPS coordinates not handled	Average readings collected within a 10-metre radius
Data retention policy not defined	Delete raw readings after 12 months

Table 2.2: Missing backend requirements and their additions

2.3.3 ML Coverage Prediction Gaps

Gap Identified	New Requirement Added
Minimum data needed for predictions not defined	ML predictions require at least 50 readings within 1 km squared
ML accuracy target not defined	ML model must achieve at least 80% classification accuracy

Table 2.3: Missing ML requirements and their additions

This analysis identified 4 inconsistencies, 7 ambiguities, and 11 missing requirements relevant to the Crowdsensed Drive Test Mobile Application MVP. Each issue was documented with its source, impact, and proposed resolution.

3. Requirement Prioritization

Not everything can be built at once. Prioritization makes sure the team focuses on the features that matter most first. We used the MoSCoW method to assign every requirement to one of four categories based on its importance and how difficult it is to implement.

Requirement prioritization is a critical activity in the software engineering lifecycle. It ensures that development efforts are directed toward the features that deliver the most value relative to their implementation complexity. This report presents the prioritized requirements for the Crowdsensed Drive Test Mobile Application developed by Group 13.

The prioritization is grounded in empirical evidence gathered during Task 2 (Requirement Gathering), including:

- A survey of 100 smartphone users across Buea
- Interviews with MTN and Orange network engineers
- Reverse engineering of OpenSignal and Network Cell Info Lite
- Team brainstorming sessions and secondary research (3GPP, ITU-D, Android Telephony API docs)

Prioritization Framework

Priority Level	Count	What it means
Must Have	5 requirements	Non-negotiable. These define the Minimum Viable Product (MVP). Without them, the app does not work.
Should Have	4 requirements	High value. These go into the v1.0 release after the MVP core is in place.
Could Have	3 requirements	Useful but not critical. Deferred to later if time and resources allow.
Won't Have (now)	3 requirements	Important long-term but out of scope for this phase.

Table 3.1: MoSCoW priority categories

Additionally, each requirement is evaluated on two axes:

- Importance: the degree to which the requirement is critical to the app's core purpose and stakeholder satisfaction

- Feasibility : the technical and resource complexity of implementing the requirement within the current project scope and timeline

Together, these two dimensions form the basis of the priority matrix presented in Section 4.

3.1 Evidence Base

The prioritization decisions are grounded in real data from Task 2:

- 56 of 100 users (56%) required full data anonymisation as a condition for participating
- 37 users (37.8%) listed minimal battery drain as a primary concern
- 36 users (36.7%) required an on/off toggle for data collection
- Average network quality rating of 2.6 out of 5, confirming a genuine coverage problem
- Engineers confirmed RSRP, SINR, and RSRQ as the core KPIs, sampled at minimum every 5 seconds
- GPS accuracy threshold of 15 metres comes from network planning standards

3.1.1 Survey Findings (100 Smartphone Users, Buea)

- 56 of 100 users (56%) required full data anonymisation as a condition for participation
- 37 users (37.8%) prioritized minimal battery usage — identified as the #2 user reluctance factor
- 36 users (36.7%) required a full On/Off toggle for data collection
- 16 users (16.3%) requested WiFi-only data upload to protect limited data plans
- Average network quality rating: 2.6–2.7 / 5, confirming a genuine coverage problem
- Network split: MTN 65%, Orange 25%, Camtel 10%
- Top problem areas: slow internet speed (89.9%), low signal strength (49.5%), rural coverage (45.5%)

3.1.2 Engineer Interview Findings (MTN & Orange)

- Core KPIs to collect: RSRP, SINR, RSRQ + GPS coordinates
- Minimum acceptable sampling interval: every 5 seconds
- GPS accuracy threshold: discard readings with error >15 metres
- Coverage hole definition: RSRP below -110 dBm
- Preferred data export format: GeoJSON and/or CSV with heatmap overlay

3.1.3 Reverse Engineering (OpenSignal, Network Cell Info Lite)

- Passive background data collection is technically viable on Android using the Telephony API
- Real-time signal dashboards improve user engagement and perceived value
- UX patterns: minimal UI during collection, clear permission prompts, toggle controls

3.2 Priority Matrix

Requirement	Category	Importance	Feasibility	Priority	Evidence
Collect RSRP, SINR, RSRQ and GPS	Functional	High	High	Must Have	Engineers and survey
Full data anonymisation	Non-Functional	High	High	Must Have	56% of users
5-second sampling interval	Functional	High	High	Must Have	Engineer interviews
On/Off toggle for collection	Functional	High	High	Must Have	36 survey users
Discard GPS readings above 15m accuracy	Functional	High	High	Must Have	Network planning standards
WiFi-only upload option	Functional	Medium	High	Should Have	16 survey users
Battery-efficient background sensing	Non-Functional	High	Medium	Should Have	37% of users
Transparent consent prompts	Non-Functional	High	Medium	Should Have	Privacy best practice
GeoJSON/CSV export and heatmap	Functional	Medium	Medium	Should Have	Engineer preference
Real-time signal dashboard	Functional	Medium	Medium	Could Have	Brainstorming
Configurable sampling interval	Functional	Low	High	Could Have	Brainstorming
Indoor/outdoor environment tagging	Functional	Medium	Medium	Could Have	Survey insights

Requirement	Category	Importance	Feasibility	Priority	Evidence
ML-based coverage prediction	Functional	High	Low	Won't Have (now)	Scope assessment
Multi-operator analytics	Functional	Medium	Low	Won't Have (now)	Scope assessment
iOS support	Non-Functional	Low	Low	Won't Have (now)	Scope assessment

Table 3.2: Full requirement priority matrix

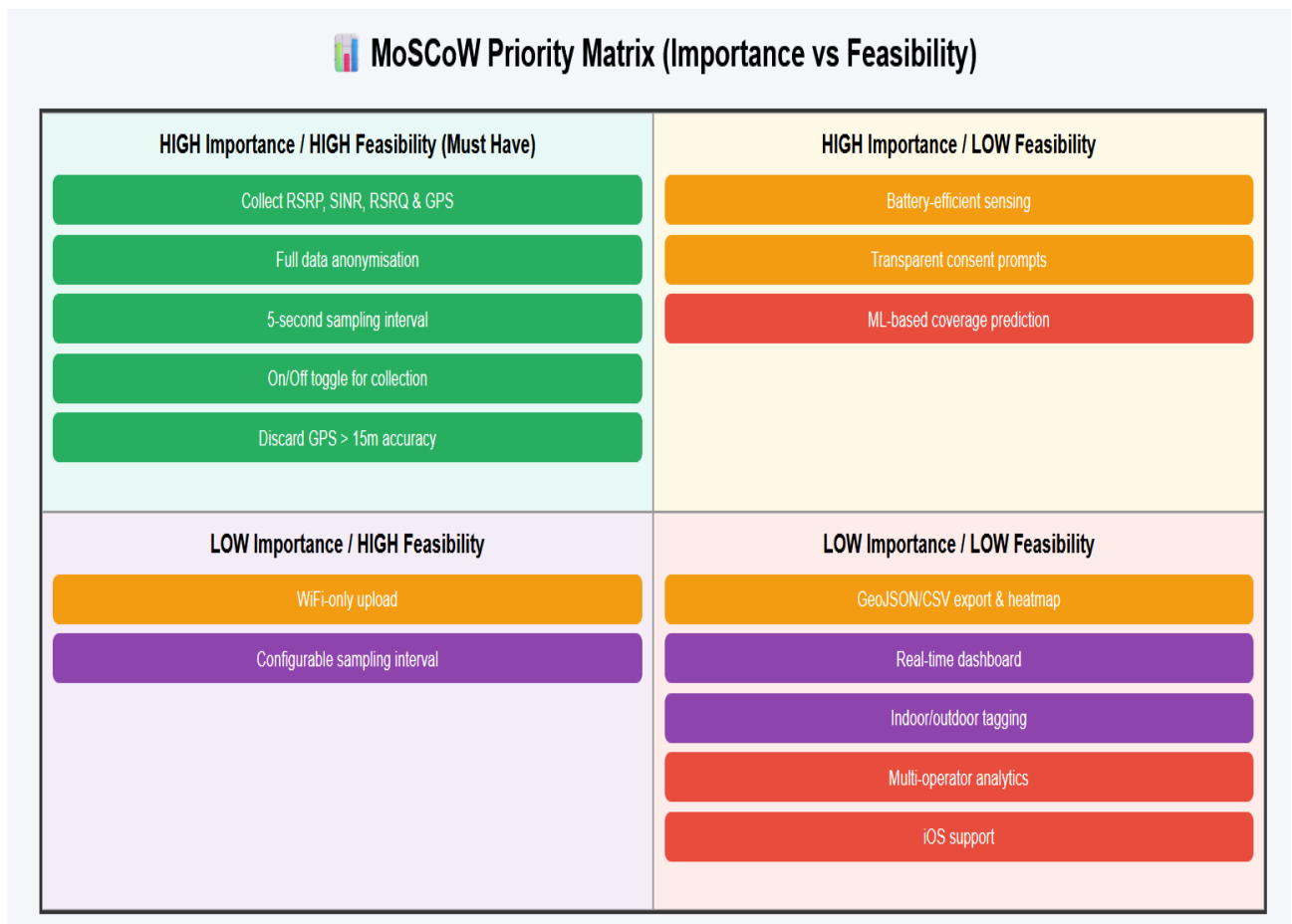


Figure 3: Requirement priority matrix visualisation

3.3 Summary by Priority

Priority	Functional	Non-Functional	Total
Must Have	3	2	5
Should Have	2	2	4
Could Have	3	0	3
Won't Have (now)	3	0	3
Total	11	4	15

Table 3.3: Requirements count by priority and category

3.4 Detailed Justification by Priority Level

3.4.1 Must Have Requirements

These five requirements form the Minimum Viable Product (MVP). All are high importance and high feasibility, meaning they are both critical to stakeholder satisfaction and straightforward to implement using the Android Telephony API.

- Collect RSRP, SINR, RSRQ + GPS; the fundamental data the entire system is built upon. Without this, the application has no purpose. Confirmed as the core KPI set by MTN/Orange engineers.
- Full data anonymization; 56% of willing users made this a hard condition. Failure to implement this would likely prevent widespread adoption and violates data privacy best practices.
- 5-second sampling interval; the industry minimum established by engineer interviews. Less frequent sampling reduces data quality below operational usefulness.
- On/Off toggle for data collection; 36 users explicitly requested user control. This is a simple UI feature that directly addresses the #4 user reluctance factor (app permissions / control anxiety).
- Discard GPS readings >15m accuracy; ensures that geospatially inaccurate records do not corrupt the coverage prediction model. Technically simple to implement via Android Location API accuracy fields.

3.4.2 Should Have Requirements

These four requirements are strongly desired but involve moderate implementation complexity. They are targeted for inclusion in the v1.0 release following the MVP core.

- WiFi-only upload; protects users with limited mobile data. Addresses reluctance factor #3 (excessive data usage). Feasible via Android NetworkCapabilities API.
- Battery-efficient background sensing; the #2 user reluctance factor. Requires careful use of Android WorkManager or JobScheduler for battery-aware background execution. High importance, moderate complexity.
- Transparent consent prompts; addresses privacy concern #1. Requires thoughtful UX design (informed consent flow, GDPR-style language). Medium complexity but essential for ethical compliance.
- GeoJSON / CSV export + heatmap; the preferred data format of operator engineers. Drives real-world utility of the system for MTN/Orange network planning teams.

3.4.3 Could Have Requirements

These enhancements are valuable but not essential for the core product. They are deferred to post-MVP iterations if time and resources allow.

- Real-time signal dashboard; improves user experience and may increase retention. Survey data shows users appreciate transparency about what is being collected. Medium feasibility due to UI rendering complexity.
- Configurable sampling interval; useful for engineers who may want finer or coarser granularity. Low priority for end users but easy to add as a settings option.
- Indoor/outdoor environment tagging; 53.5% of users reported indoor signal problems. While valuable, automatic environment detection requires sensor fusion (accelerometer, barometer, WiFi) adding significant complexity.

3.4.4 Won't Have (Current Phase)

These requirements are acknowledged as important for the project's long-term vision but are explicitly out of scope for this phase due to low feasibility or dependency on prior phases.

- ML-based coverage prediction model; the ultimate goal of the system, but requires substantial training data from a fully operational crowdsensing pipeline first. Cannot be built before data collection is working.
- Multi-operator comparative analytics; useful for comparing MTN vs. Orange coverage. Requires a more mature backend with operator tagging and significant data volume.
- iOS support; the team's reverse engineering focused on Android. iOS introduces a separate development stack (Swift/Kotlin vs. Java), significantly extending scope beyond the current project timeline.

4. Requirement Classification

Requirements are classified into two types: functional requirements describe what the system must do, and non-functional requirements describe how well the system must perform. A total of 16 requirements were identified: 8 functional and 8 non-functional.

We gathered the requirements through surveys (100 smartphone users in Buea), interviews with MTN and Orange network engineers, team brainstorming sessions, and reverse engineering of comparable applications such as OpenSignal and Network Cell Info Lite. This report classifies those requirements into functional and non-functional categories.

Classification Framework

Requirements are classified according to the standard software engineering distinction between functional and non-functional requirements:

- **Functional Requirements (FR):** Define what the system must do in other words, specific behaviours, features, and operations the application must perform.
- **Non-Functional Requirements (NFR):** Define how the system performs like quality attributes such as performance, security, usability, and reliability.

4.1 Functional Requirements

These are the specific things the app must be able to do. Each one is tied back to a source from Task 2.

Requirement	Description	Source
Collect signal metrics (RSRP, SINR, RSRQ and GPS)	App passively collects LTE signal parameters and GPS coordinates during use	MTN and Orange network engineers
Configurable sampling interval (default: 5 seconds)	System samples network data every 5 seconds; interval is user-adjustable	Engineer minimum threshold
Filter inaccurate GPS readings (above 15m accuracy)	Discard GPS readings with accuracy worse than 15 metres	Network planning standards

Requirement	Description	Source
Anonymise all collected user data	All transmitted data must strip personally identifiable information before upload	56 of 100 survey users
On/Off toggle for data collection	Users can start and stop data collection at any time from within the app	36 survey users
WiFi-only upload option	Users may restrict data uploads to WiFi connections only	16 survey users
GeoJSON / CSV export with heatmap visualisation	Data exportable in GeoJSON or CSV; app renders a coverage heatmap	Operator engineers
Transparent consent prompts	App clearly informs users what is collected and obtains explicit consent before collection begins	Privacy best practices and survey feedback

Table 4.1: Functional requirements with sources

4.2 Non-Functional Requirements

These define the quality standards the system must meet. They were drawn from user privacy expectations, technical benchmarks from engineers, and operational constraints.

Category	Requirement	Rationale
Privacy	Full data anonymisation before transmission	57% of surveyed users would only share data anonymously
Performance	Battery-efficient background sensing	Battery drain was the second biggest concern among reluctant users (37.8%)
Usability	Minimal user interaction during collection	Passive collection is the core design principle

Category	Requirement	Rationale
Reliability	GPS accuracy threshold of 15 metres or less	Inaccurate GPS makes coverage maps unusable for network planning
Data Quality	Minimum sampling rate of 5 seconds	Engineers confirmed this as the minimum acceptable threshold for meaningful KPI data
Security	Informed consent before any data collection begins	Regulatory compliance and ethical data collection practice
Scalability	System must aggregate data from large numbers of concurrent users	Crowdsensing only works at scale
Compatibility	Android platform support using TelephonyManager API	Target users are predominantly on Android; API access is required for signal metrics

Table 4.2: Non-functional requirements with rationale

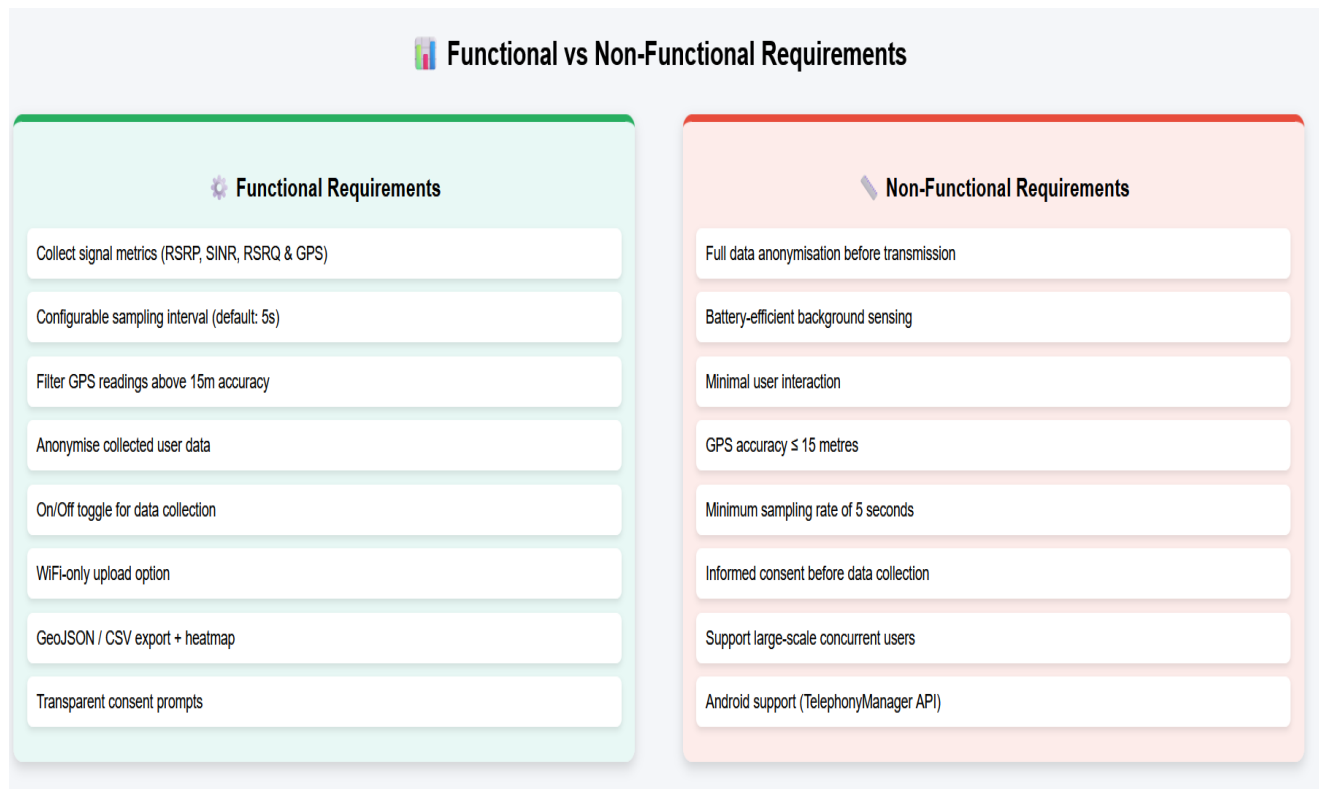


Figure 4: Functional vs non-functional requirement classification overview

5. Software Requirements Specification (SRS)

CrowdSenseNet

Design and Implementation of a Crowdsensed Drive Test Mobile Application for Coverage Prediction

5.1 Introduction

5.1.1 Purpose

This document provides the complete Software Requirements Specification (SRS) . The purpose of this document is to specify the requirements for **CrowdSenseNet**, a system designed to address the lack of transparent network coverage data in Cameroon. By leveraging crowdsensing technology, the system aims to empower both consumers and network operators with real-time, accurate signal maps, replacing expensive traditional drive-tests with a community-driven mobile solution.”

5.1.2 Document Conventions

- **Must Have:** Critical requirements for the Minimum Viable Product (MVP)
- **Should Have:** Important features that add significant value but are not blocking
- Signal measurements are expressed in **dBm** for power and **dB** for quality

5.1.3 Intended Audience

- **Developers and architects:** focus on Sections 5.3 (System Features) and 5.7 (Architecture)
- **Network engineers:** focus on Section 5.1.5 (Definitions) and Section 5.6 (Data Requirements)
- **Academic supervisors:** focus on Section 6 (Stakeholder Validation) and the traceability matrix at the end

5.1.4 Project Scope

CrowdSenseNet is an Android application that turns ordinary smartphones into **Class-2 IoT sensing devices**. It passively collects cellular metrics (RSRP, RSRQ, SINR) to build real-time coverage heatmaps and predict coverage holes (RSRP below -110 dBm) using a machine learning model. **iOS support is excluded** in this phase due to API restrictions and scope limitations.

The Minimum Viable Product (MVP) covers: signal data collection (RSRP, RSRQ, SINR, GPS), local offline storage, backend data upload, a basic coverage map, and ML-based coverage hole prediction.

5.1.5 Key Definitions

TERM	DEFINITION
RSRP	Reference Signal Received Power. The primary LTE signal strength metric. Coverage hole threshold: below -110 dBm
RSRQ	Reference Signal Received Quality. Measures signal quality relative to interference
SINR	Signal to Interference-plus-Noise Ratio. Indicates signal clarity
COVERAGE HOLE	Any location where predicted RSRP is below -110 dBm
UUID	Universally Unique Identifier. A random anonymous token generated on first launch
CROWDSENSING	Collecting data passively from many users to build a picture of real-world conditions
DRIVE TEST	Traditional method of measuring mobile coverage using engineers with specialised equipment in a vehicle

Table 5.1: Key definitions and terminology

5.2 Overall Description

5.2.1 Product Perspective

CrowdSenseNet is a standalone Android application operating within a three-tier architecture:

- **Tier 1 (Client):** Android app written in Kotlin for data collection and local SQLite storage
- **Tier 2 (Backend):** Python FastAPI server for data validation and PostgreSQL storage
- **Tier 3 (Analytics):** Python ML model for coverage prediction and heatmap generation

5.2.2 User Classes

- **General Contributors:** Non-technical users who passively contribute signal data.
- **Network Engineers:** High-technical users at MTN/Orange who access maps to fix problem areas.
- **System Administrators:** Experts managing the backend, database, and ML updates.

5.2.3 Operating Environment

- **Mobile platform:** Android 8.0 (API level 26) minimum
- **Local database:** Room (SQLite-based)
- **Remote database:** PostgreSQL with PostGIS for spatial data
- **Network:** 4G, 5G, or WiFi for data synchronisation

5.2.4 Constraints

- **Battery constraint:** Adaptive sampling is required to prevent unacceptable drain
- **Geographic scope:** Limited to the Cameroon bounding box (Latitude 1.65N to 13.08N)
- **Permission dependency:** Requires ACCESS_FINE_LOCATION and READ_PHONE_STATE permissions

5.2.5 Assumptions and Dependencies

- Users must grant required hardware permissions on first launch
- A minimum of 50 readings per 1 km squared is required before the ML model can generate a prediction
- Android TelephonyManager API must support SINR on the user's device (varies by Android version and chipset)

5.3 System Features

5.3.1 Functional Requirements

ID	Requirement	Priority
FR-01	Single-tap session start/stop from the Home screen	Must Have
FR-02	Auto-pause collection when the device is stationary for more than 30 seconds	Must Have
FR-03	Auto-resume collection when device speed exceeds 5 km/h	Must Have
FR-04	Capture RSRP, RSRQ, SINR, and RSSI via TelephonyManager API	Must Have
FR-05	Capture GPS coordinates with every signal reading	Must Have
FR-06	Discard readings where GPS accuracy is worse than 15 metres	Must Have
FR-07	Store all readings immediately in local Room database	Must Have
FR-08	Generate and store a random anonymous UUID on first launch	Must Have
FR-09	Upload data to server over WiFi (default)	Should Have
FR-10	Allow user to enable mobile data upload via Settings toggle	Should Have
FR-11	Retry failed uploads automatically every 30 minutes	Should Have
FR-12	Export collected data in GeoJSON and CSV formats	Should Have
FR-13	Render a coverage heatmap on the Map screen	Should Have

FR-14	Display a real-time signal dashboard on the Home screen	Could Have
FR-15	Allow user to adjust sampling interval (5 to 30 seconds) in Settings	Could Have
FR-16	Tag readings as indoor or outdoor based on sensor data	Could Have

Table 5.3: Functional requirements with IDs and priorities

5.4 External Interface Requirements

5.4.1 User Interface

- **Modern Design:** Built with Material Design 3 and responsive ConstraintLayouts
- **Four core screens:** Home (Dashboard), Map (Heatmap), History, and Settings
- All key actions must be reachable within 3 taps
- Supports English and French based on device language settings

5.4.2 Hardware Interfaces

- **GPS:** FusedLocationProvider API for battery-efficient location tracking
- **Cellular modem:** TelephonyManager for hardware-level signal parameter access

5.4.3 Software Interfaces

- **Maps:** Google Maps API or OpenStreetMap for the heatmap layer
- **Server API:** REST over HTTPS for all app-to-server communication
- **Local DB:** Room (Android) linked to SQLite for offline storage

5.4.4 Communication Interfaces

- All transmissions must use TLS 1.2 or higher
- WiFi-only upload is the default; mobile data upload is opt-in via Settings

5.5 Non-Functional Requirements

5.5.1 Performance

- App must launch and reach the Home screen within 3 seconds
- Heatmap must render within 4 seconds on a stable connection

5.5.2 Resource Constraints

- Background collection must consume less than 5% battery per hour

- Local storage is capped at 50MB with automatic deletion of synced records

5.5.3 Security

- The app must not collect name, email, IMEI, or IP address
- Backend API endpoints must require authentication tokens
- All data transmissions must be encrypted using TLS 1.2 or higher

5.5.4 Reliability

- No data loss during app crashes: all readings are written to local DB immediately
- Failed uploads are retried automatically every 30 minutes

5.5.5 Usability

- English and French language support based on device settings
- No more than 3 steps to enable or disable data collection

5.5.6 ML Accuracy

- The ML model must achieve at least 80% classification accuracy for coverage hole prediction
- Predictions require a minimum of 50 readings within 1 km squared to be valid

6. Validation with Stakeholders

6.1. Introduction

When you gather requirements from stakeholders, you end up with a list of things the system should do. But writing down those requirements is not enough on its own. You need to go back to the people involved and confirm that what you wrote actually matches what they meant. That process is called validation.

Validation is about making sure the requirements you gathered are correct, complete, and acceptable to everyone involved. It checks that the system you plan to build will actually meet real needs, follows the rules and regulations, and is realistic to implement. Without validation, requirements stay as guesses. With it, they become a solid base to build on.

For the Crowdsensed Drive Test Mobile Application, validation is especially important because the app involves data collection from real users in Cameroon. That means users, telecom operators, regulators, and supervisors all have a say in whether the requirements make sense.

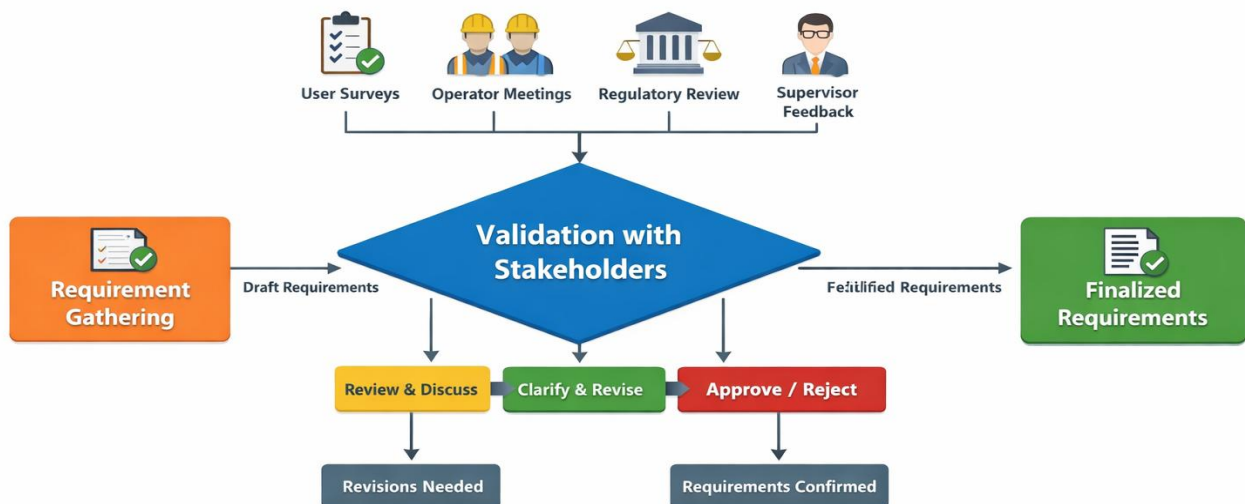


Figure 1: Overview of the stakeholder validation process flow

6.2. Why Validation is Important

There are four main reasons why validation is a step you cannot skip:

- **Accuracy:** It confirms that the requirements actually reflect what stakeholders said they need, not what the team assumed.

- **Feasibility:** It checks that the requirements can be built using the tools and technology available. For example, can Android actually give us access to SINR and GPS data? Yes, it can, so that requirement is feasible.
- **Trust:** It builds confidence among users, telecom operators, and supervisors that the system will do what it promises.
- **Compliance:** It makes sure the system follows legal and regulatory rules. For example, data anonymization has to meet Cameroon's telecom privacy standards before the app can collect any user data.

A simple example: if users say they need their data to stay anonymous, validation means going back and confirming with both the users and the regulators that removing names and phone numbers and storing only network parameters is enough. If it is not, the requirement needs to change before any design work begins.

6.3. Stakeholders Validation Methods

Different stakeholders have different concerns, so each group needs its own validation approach. The table below summarizes how each group was involved in the validation process:

Stakeholder	Method Used	Validation Example
Users (Smartphone Owners)	Follow-up surveys, prototype testing, feedback sessions	Show prototype with ON/OFF toggle. Confirm it meets their privacy expectations.
Mobile Network Operators (MTN, Orange, Nexttel)	Requirement review meetings with engineers	Confirm that RSRP, SINR, RSSI are useful for coverage prediction and match their standards.
Regulators (Telecom Authorities)	Compliance checks against national data protection laws	Present anonymization strategy and confirm it aligns with Cameroon's telecom regulations.
Supervisors (Lecturers)	Academic review of documentation and methodology	Ask if functional vs non-functional requirement classification follows proper software engineering.

Table 1: Validation methods used for each stakeholder group

Each group listed above plays a different role in the project, so the questions you ask them during validation are completely different. Users care about privacy and ease of use. Telecom engineers care about whether the data collected is actually useful for their work. Regulators care

about legal compliance. And supervisors are checking whether the software engineering methodology is sound.

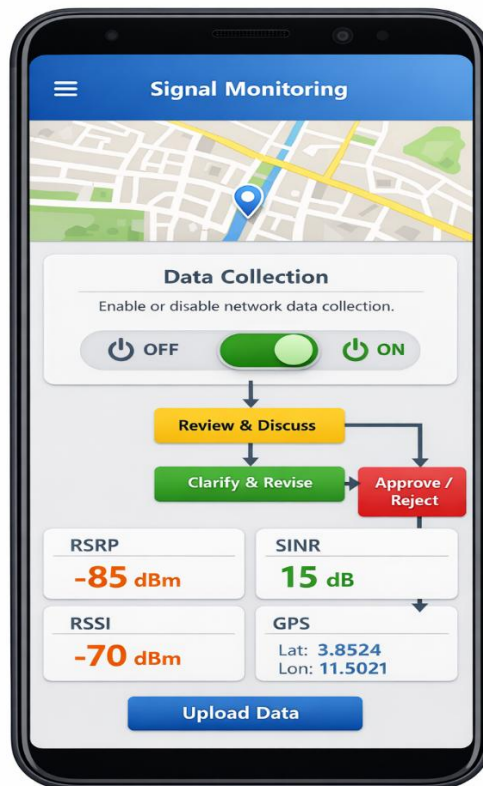


Figure 2: App prototype showing ON/OFF data collection toggle for user validation testing

6.4. Validation Techniques

There are four main techniques used to validate requirements with stakeholders. Each one works better in different situations:

- **Requirement Review Meetings:** You sit down with stakeholders, present the requirements, and ask for their feedback. This is the most direct way to catch anything that is unclear or missing.
- **Prototyping:** You build a simple version of the app and let users interact with it. This is more effective than just describing the app in writing, because people can see and feel what you are building. For this project, a basic prototype of the ON/OFF data collection toggle was shown to users to confirm it felt intuitive and private enough.
- **Walkthroughs:** You explain each requirement one by one to stakeholders, going through the logic step by step. This helps catch contradictions or gaps that are easy to miss when reading a document.

- **Checklists:** You use a structured list of questions to make sure each requirement is clear, complete, feasible, testable, and accepted by the right people. See Section 5 for the full checklist used in this project.

6.5. Validation checklist

The checklist below was used to evaluate each major requirement before it was accepted. Every requirement had to pass all five criteria before it could be treated as final. The table shows one example for each criterion:

Validation Criterion	Question to Ask	Example Outcome
Clarity	Is the requirement free of ambiguity?	"Collect RSRP every 5 minutes" is clear. "Collect data regularly" is vague.
Completeness	Does it cover all stakeholder needs?	Users want anonymity, so requirement added: "Data must be anonymized."
Feasibility	Can it be implemented with current tools?	Android APIs allow access to SINR and GPS, so yes, this is feasible.
Testability	Can it be verified during testing?	Battery usage of 5% per hour can be measured during device trials.
Acceptance	Do stakeholders agree with the requirement?	MTN engineers confirm usefulness of parameters. Users confirm privacy toggle.

Table 2: Validation checklist with example outcomes for each criterion



Figure 3: Visual summary of the validation checklist criteria

Each requirement in the final list for the Crowdsensed Drive Test Mobile Application was checked against these five criteria. Any requirement that failed even one of them was sent back for revision before being finalized. This process is what turns a rough list of stakeholder wishes into proper, testable system requirements.

6.6 Validation Outcomes

Running every requirement through this checklist produced the following key confirmations:

- Users confirmed they need anonymity and a clear ON/OFF control. These are now locked-in non-negotiable requirements.
- Telecom engineers confirmed that RSRP, SINR, and RSSI are the right parameters to collect and that they match the parameters used in their own internal systems.
- The anonymisation approach (removing all PII and using only a random UUID) was found to be acceptable under the relevant regulatory expectations.
- Academic supervisors confirmed that the functional and non-functional classification follows standard software engineering methodology.

CONCLUSION

This report has taken the raw requirements gathered in Task 2 and turned them into a solid, well-structured foundation for building CrowdSenseNet. The process was systematic: we reviewed every requirement for completeness, clarity, technical feasibility, and dependencies. We resolved four inconsistencies, clarified seven ambiguous requirements, and added eleven requirements that were missing entirely.

We then prioritised all requirements using the MoSCoW method, giving the development team a clear order in which to work. The five Must Have requirements define the MVP. The four Should Have requirements complete the v1.0 feature set. Everything else is clearly deferred or excluded with reasons given.

The classification exercise confirmed that the system has 8 well-defined functional requirements and 8 quality-focused non-functional requirements, all grounded in real evidence from stakeholders. The SRS formalises all of this into a document that developers, engineers, and supervisors can all read and work from.

Finally, validation with stakeholders confirmed that the requirements are correct, feasible, legal, and accepted by the people who will use and oversee the system. This step is what separates a guessed set of requirements from a confirmed one.

The team can now move into the design phase with confidence that what is being built is the right thing, built the right way, for the right people.